

BRANCHING OUT: A DIVERSITY BONUS FOR EXPLORATION IN LLM-POWERED TREE SEARCH

Anonymous authors

Paper under double-blind review

ABSTRACT

Large Language Model (LLM) solvers guided by tree search algorithms often struggle with exploration, getting trapped in local optima on complex reasoning tasks like the Abstraction and Reasoning Corpus (ARC). To address this, we introduce a diversity bonus into the tree search scoring function to explicitly reward novel solutions. We evaluate two heuristics for measuring novelty: syntactic similarity of generated Python code and semantic similarity of the resulting output grids. This bonus is blended with the public score via a hyperparameter α . Experiments on the ARC benchmark show that a standard MCTS baseline achieves 0.31 test accuracy. By adding a code similarity bonus with $\alpha = 0.3$, we significantly improve accuracy to 0.37. We find that other configurations, such as a higher α or using output-based similarity, are less effective, highlighting that syntactic diversity is a particularly potent heuristic for this task. Our results demonstrate that explicitly promoting solution diversity is an effective strategy for guiding LLM-powered program synthesis towards more generalizable solutions.

1 INTRODUCTION

While combining Large Language Models (LLMs) with search algorithms like Monte Carlo Tree Search (MCTS) shows promise for complex reasoning tasks (Yuichi Inoue, 2025), this approach faces a critical challenge: a lack of exploration. On benchmarks like the Abstraction and Reasoning Corpus (ARC), which demand inference of complex rules from few examples, the search often becomes trapped in *local optima*. Guided by scores from public examples, the LLM repeatedly generates minor variations of a single, flawed idea, stifling the discovery of novel solution strategies.

To address this, we propose integrating a diversity bonus into the tree search’s scoring mechanism. This bonus explicitly rewards solutions that are dissimilar to those already explored, encouraging the search to venture into new regions of the solution space. We investigate two heuristics for this bonus: one based on the syntactic Jaccard similarity of generated Python code, and another on the semantic similarity of resulting output grids. A hyperparameter, α , controls the balance between exploiting high-scoring solutions and exploring novel ones.

Experiments on the ARC benchmark validate our approach. While a standard MCTS baseline achieves 0.31 test accuracy, incorporating a code similarity bonus with $\alpha = 0.3$ significantly boosts accuracy to 0.37. Our results show this syntactic bonus is more effective than a semantic, output-based one, which reached a peak accuracy of 0.34. The choice of α is also critical, as a higher weight for the code bonus decreased performance to 0.32, underscoring the need for a careful balance between exploration and exploitation. These findings demonstrate that explicitly encouraging syntactic diversity is a robust strategy for guiding tree search towards generalizable solutions.

The main contributions of this work are as follows:

- We identify the problem of limited exploration in LLM-powered tree search for abstract reasoning, where solvers often converge to local optima.
- We propose a diversity bonus integrated into the search’s scoring function, and introduce two specific heuristics for it: syntactic code similarity and semantic output similarity.
- We provide strong empirical evidence on the ARC benchmark that our method significantly improves performance, with code-based diversity yielding the largest gains.

- We analyze the impact of the diversity weight, α , showing that a moderate value is crucial for balancing exploration and exploitation.

2 RELATED WORK

LLMs with Tree Search Combining Large Language Models (LLMs) with search algorithms like Monte Carlo Tree Search (MCTS) is a potent strategy for complex reasoning tasks (Yuichi Inoue, 2025). In this paradigm, the LLM generates candidate steps while the search algorithm explores the solution space. The Tree of Thoughts (ToT) framework, for example, uses an LLM to explore diverse reasoning paths and evaluate intermediate steps to guide its search (Yao et al., 2023). While such methods also aim to improve exploration, our approach is distinct. Rather than engineering a specific reasoning structure like ToT, we introduce a general and lightweight modification to the search’s scoring function. Our diversity bonus directly incentivizes novelty among complete solutions, making it a broadly applicable heuristic for any tree-search-based solver.

Diversity in Generative Models and Search Promoting diversity is a known challenge in sequence generation, where standard decoding methods like beam search often yield repetitive outputs. Techniques like Diverse Beam Search address this by penalizing similarity among hypotheses within the beam during a single decoding step (Vijayakumar et al., 2016). Our work shares the goal of encouraging diversity but operates at a higher level of abstraction. Instead of influencing token-level choices within a single generation, our diversity bonus acts as a meta-heuristic guiding the entire tree search over complete, executable solutions. This allows us to shape the high-level exploration strategy rather than just the low-level text generation process.

3 BACKGROUND

We evaluate our method on the Abstraction and Reasoning Corpus (ARC), a challenging benchmark for abstract reasoning. Each ARC task requires a solver to infer a transformation rule from a few input-output grid pairs and apply it to an unseen test input. The core challenge is few-shot generalization and program synthesis from visual examples, which makes it a difficult test for modern AI systems.

Our work builds on Monte Carlo Tree Search (MCTS), a heuristic search algorithm that balances exploration and exploitation. MCTS iteratively builds a search tree via four steps: Selection, Expansion, Simulation, and Backpropagation. The selection step is guided by the Upper Confidence bound for Trees (UCT) score (Kocsis & Szepesvari, 2006), which selects child nodes to expand. We use a UCT variant common in LLM applications:

$$\text{UCT}(v) = Q(v) + c \cdot P(v) \sqrt{\frac{\ln N(p)}{N(v)}} \quad (1)$$

where $Q(v)$ is the node’s average reward, $P(v)$ its prior probability, $N(p)$ and $N(v)$ are parent and node visit counts, and c is an exploration constant. After a leaf node is expanded, a simulation estimates its value, which is then backpropagated to update the path to the root.

In modern implementations, LLMs are integrated into the MCTS framework to act as powerful generators for the simulation step (Yuichi Inoue, 2025). Instead of a random rollout, the LLM is prompted with the current node’s state (e.g., the problem description and prior attempts) to generate a new candidate solution. This candidate is evaluated, and its score is backpropagated. This hybrid approach combines the LLM’s generative capabilities with the systematic exploration of tree search. Our work enhances this paradigm by introducing a mechanism to explicitly improve exploration, addressing the tendency of these solvers to get stuck in local optima.

4 METHOD

To promote exploration, we modify the scoring function used to guide the MCTS algorithm. Instead of relying solely on the public score, S_{public} —the fraction of demonstration examples a solution solves—we introduce a blended score, S_{blended} , that incorporates a novelty bonus:

$$S_{\text{blended}} = (1 - \alpha)S_{\text{public}} + \alpha S_{\text{novelty}} \quad (2)$$

where S_{novelty} is a bonus rewarding solutions that are dissimilar to previously seen ones, and $\alpha \in [0, 1]$ is a hyperparameter that balances exploitation (S_{public}) and exploration (S_{novelty}).

The novelty bonus is derived from a similarity metric. For a newly generated candidate solution c and a history of previously generated solutions H , the bonus is calculated as:

$$S_{\text{novelty}}(c, H) = 1 - \max_{h \in H} \text{sim}(c, h) \quad (3)$$

If the history H is empty, S_{novelty} is defined as 1.0. We propose and evaluate two different similarity functions, $\text{sim}(\cdot, \cdot)$.

Syntactic Diversity (Code Similarity) This heuristic measures the novelty of a solution based on its source code. For two Python code snippets, c_1 and c_2 , their similarity is computed using the Jaccard index of their token sets. The code is first tokenized by splitting on non-alphanumeric characters and converting to lowercase. Common English stopwords are removed to focus on meaningful tokens. The Jaccard similarity is then:

$$\text{sim}_{\text{code}}(c_1, c_2) = \frac{|\text{tokens}(c_1) \cap \text{tokens}(c_2)|}{|\text{tokens}(c_1) \cup \text{tokens}(c_2)|} \quad (4)$$

Semantic Diversity (Output Similarity) This heuristic measures novelty based on the functional behavior of the code. We execute a candidate solution on the first demonstration input to generate an output grid. The similarity between two grids, g_1 and g_2 , is the fraction of cells with identical values. As a special case, if the grids have different dimensions, their similarity is defined as 1.0. This yields a novelty bonus of 0, penalizing solutions that produce outputs with previously unseen dimensions.

Integration with MCTS At the beginning of a run, we initialize an empty history list, H . During the MCTS process, when a new node is generated via LLM simulation, we first evaluate its corresponding solution on the public demo examples to get S_{public} . We then compute S_{novelty} by comparing the solution (either its code or its output grid) against all items currently in H . The final S_{blended} is calculated using Equation (2). This blended score is then backpropagated up the tree to update the value estimates of ancestor nodes and is subsequently used in the UCT calculation (Equation (1)) for the selection phase. Finally, the representation of the new solution is added to H for future novelty calculations.

5 EXPERIMENTAL SETUP

We evaluate our method on the Abstraction and Reasoning Corpus (ARC) using a standard MCTS framework. For each task, we run the search for a fixed budget of 16 node expansions. The generative step uses the `openrouter_deepseek_deepseek-chat-v3.1` LLM with a sampling temperature of 0.6. We compare five configurations: a **Baseline** MCTS with no diversity bonus ($\alpha = 0.0$); two **Code Diversity** variants using syntactic similarity with $\alpha = 0.3$ and $\alpha = 0.5$; and two **Output Diversity** variants using semantic similarity with $\alpha = 0.3$ and $\alpha = 0.5$.

We evaluate performance using three metrics. **Test Accuracy (Coverage)** is the fraction of tasks where at least one solution passes the hidden test case. **Perfect Accuracy (Coverage)** is a stricter version, requiring a solution to pass both public and private tests. Finally, **Pass@2** measures the private test accuracy of the better of the top two solutions ranked by public score, assessing how well the public score correlates with private test success.

6 RESULTS

Our experiments show that incorporating a diversity bonus significantly improves performance. Table 1 summarizes the final test accuracies, showing that code-based diversity with $\alpha = 0.3$ achieves the best result (0.37), a substantial gain over the baseline’s 0.31.

The detailed performance throughout the search reinforces these findings. The code diversity method with $\alpha = 0.3$ consistently outperforms all other configurations across all metrics, finding correct solutions more efficiently. Our results also highlight the importance of the diversity heuristic and

Table 1: Final test accuracy for different diversity strategies. The code diversity method with $\alpha = 0.3$ achieves the best performance.

Method	α	Test Accuracy
Baseline (No Diversity)	0.0	0.31
Code Diversity	0.3	0.37
Code Diversity	0.5	0.32
Output Diversity	0.3	0.32
Output Diversity	0.5	0.34

its weighting. Syntactic (code) diversity is more effective than semantic (output) diversity, and it is sensitive to the choice of α ; increasing it to 0.5 degrades performance. In contrast, output diversity benefits from a higher α . By exploring a more diverse set of solutions, our method improves the correlation between public scores and private test success.

Limitations Our study is limited by the scope of hyperparameters and diversity heuristics tested. The findings are specific to the ARC benchmark and the chosen LLM, and may not generalize to other domains or models without further investigation. A more exhaustive search could reveal even better configurations.

7 CONCLUSIONS AND FUTURE WORK

We addressed the problem of limited exploration in LLM-powered tree search by introducing a diversity bonus to the scoring function. Our experiments on the ARC benchmark showed this simple addition to be highly effective: a bonus based on syntactic code similarity increased test accuracy from 0.31 to 0.37. This result demonstrates that explicitly encouraging solution diversity is a powerful strategy for helping search algorithms escape local optima and discover more generalizable solutions.

Future work could explore more sophisticated diversity metrics, such as those based on abstract syntax trees, or adaptive scheduling for the weighting hyperparameter α to dynamically balance exploration and exploitation. Applying this framework to other complex reasoning domains, like mathematical theorem proving, would also be a valuable test of its generality.

This work was generated by THE AI SCIENTIST.

REFERENCES

- Levente Kocsis and Csaba Szepesvari. Bandit based monte-carlo planning. pp. 282–293, 2006.
- Ashwin K. Vijayakumar, Michael Cogswell, Ramprasaath R. Selvaraju, Q. Sun, Stefan Lee, David J. Crandall, and Dhruv Batra. Diverse beam search: Decoding diverse solutions from neural sequence models. *ArXiv*, abs/1610.02424, 2016.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, T. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *ArXiv*, abs/2305.10601, 2023.
- Yuki Imajuku So Kuroki Taishi Nakamura Takuya Akiba Yuichi Inoue, Kou Misaki. Wider or deeper? scaling llm inference-time compute with adaptive branching tree search. *arXiv preprint arXiv:2503.04412*, 2025.